# Haneet Singh

Field-installed accessibility systems, hand-built hardware, and the analysis behind both.

| | |
|---|---|
| PROGRAM | BASc Mechanical Engineering, Co-op |
| TERM | 1A · Class of 2031 |
| TEAM | Waterloo Rocketry, Propulsion |

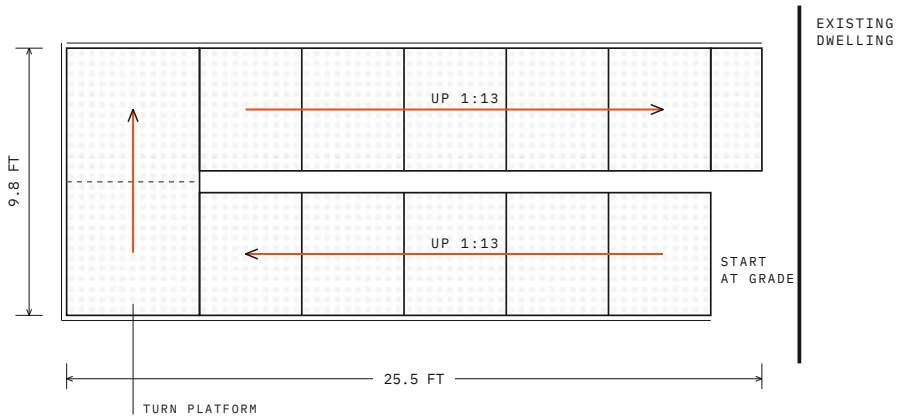



**FIG. 0.1** — Switchback ramp, plan view. 36 in rise, ~39 ft of ramp folded into a 25.5 × 9.8 ft footprint. Schematic, redrawn from the layout sheet. SEE SHEET 03

INDEX **DRAWING LIST**


| | | |
|---|---|---|
| 01 | **Profile, key results and working principles** | p. 02 |
| 02 | **Field service** · Action Medical Home Health | p. 03 |
| 03 | **Accessibility ramp systems** · field project | p. 04–05 |
| 04 | **Raspberry Pi arcade joystick** · hardware and software | p. 06–07 |
| 05 | **Arduino rover** · embedded C++ and motor control | p. 08–09 |
| 06 | **ESP32 walkie-talkie** · timed team build | p. 10 |
| 07 | **Personal finance and portfolio model** · analysis | p. 11 |


CONTACT


| | |
|---|---|
| EMAIL | h365sing@uwaterloo.ca |
| PORTFOLIO | haneet10.github.io |
| LINKEDIN | /in/haneet-singh-uw |
| GITHUB | github.com/Haneet10 |

# Engineering where it *has consequences.*

I'm a first-year Mechanical Engineering student at the University of Waterloo. Before my first lecture, I'd already spent more than a year doing technical work that people depend on: in clients' homes, in hospitals and long-term care, and on the bench.

Through my last year of high school I worked as a junior technician for a home-healthcare equipment company. Every week I serviced and assembled more than thirty wheelchairs, walkers and scooters, went out on around five site calls to install hospital beds and ceiling lifts, and helped configure modular wheelchair ramps that had to meet code on properties never designed for them. Alongside that work I build hardware: a rover my partner and I rebuilt from scratch after its controller was destroyed, an arcade joystick made from a door stop and four nails, and a walkie-talkie my team got working before anyone else in the room.

This set documents that work the way an engineer would want to read it: the brief, the constraints, the decisions and why I made them, what failed, how I found out, and what I'd change in the next revision. Every number in it comes from the job or the build itself.

## A WHAT I BRING

**Field judgment**
Measuring real sites, working to code and installing equipment that people rely on every day. → 02, 03

**Build and debug**
Soldering, machining, wiring and code, debugged together as one system rather than one part at a time. → 04, 05, 06

**Quantitative modelling**
Slope and run calculations, PWM calibration, compound growth and amortization. → 03, 05, 07

## B KEY RESULTS

| 8 | 30+ | 1st | <½ | <1 wk | 98.8% |
|---|---|---|---|---|---|
| ACCESSIBILITY RAMP SYSTEMS CONFIGURED, 12–26 FT | MOBILITY DEVICES SERVICED PER WEEK | TEAM TO A WORKING WALKIE-TALKIE | THE ORIGINAL TIME TO REBUILD THE ROVER | TO FINISH A TWO-WEEK JOYSTICK BUILD | TOP-SIX HIGH SCHOOL AVERAGE |

## C HOW I WORK

| PRINCIPLE | WHERE IT CAME FROM | SHEET |
|---|---|---|
| **Measure before deciding** | A ramp's length is fixed by its rise. The tape measure settles the design before any part is ordered. | 03 |
| **Debug the system, not the part** | A debounce setting in software changed how a wooden joystick felt in the hand. | 04 |
| **Fix it in the design when you can't swap the part** | Mismatched motors and poor wheels, corrected in calibration instead of replaced. | 05 |
| **Under a deadline, optimise for speed of change** | A breadboard beat solder because every fix took seconds. | 06 |

## D NOW

**Waterloo Rocketry, Propulsion**
Joined the propulsion sub-team of Waterloo's high-power rocketry design team in 2026, where aerospace is heading next for me.

**Looking for**
A co-op term in mechanical design, aerospace, mechatronics, manufacturing or field engineering, where I can be useful from the first week and learn from engineers who build real things.

> *"Engineering is judged by whether it works for the person using it, not by how it looks on the layout sheet."*

02 — EXPERIENCE

# Field Service & Operations

More than a year keeping mobility and medical equipment safe in homes, hospitals and long-term care.



FIG. 2.1 — Ramp modules, delivered

| EMPLOYER | TITLE | DATES |
|---|---|---|
| Action Medical Home Health | Junior Mobility Systems Technician | Jul 2025 – Sep 2026 |

| 30+ | ~5 | 8 | $10k+ |
|---|---|---|---|
| WHEELCHAIRS, WALKERS AND SCOOTERS PER WEEK | ON-SITE SERVICE CALLS PER WEEK | RAMP SYSTEMS CONFIGURED (SHEET 03) | WEEKLY BUSINESS SUPPORTED AT PEAK |

## 01 THE WORK

**01 Bench service**
Maintained, repaired and assembled more than 30 wheelchairs, walkers and scooters a week, keeping equipment safe and reliable and minimizing the time clients were without it.

**02 Site calls**
Around five on-site calls a week, installing and servicing hospital beds, ceiling lifts and accessibility equipment in homes, hospitals and nursing homes, to Ontario healthcare and long-term care standards.

**03 Ramp systems**
Configured 8 modular accessibility ramps spanning 12–26 ft and coordinated footing placement and site integration. Documented in full on Sheet 03.

**04 Billing and sales support**
Prepared invoices, quotations and service records for equipment sales, repairs and installations, supported equipment recommendations and installation plans, in a business that handled over $10,000 in its busiest weeks.

## 03 ALSO

**Purple Bean Media**, Graphic Design Co-op Student, Jun – Aug 2025. Designed social posts, stories and short videos for client accounts within established brand guidelines, managing several deadlines at once in Canva, Photoshop and Premiere Pro.

## 02 WHY IT MATTERS TO AN ENGINEERING EMPLOYER

| ON THE JOB | TRANSFERS AS |
|---|---|
| **Equipment people's safety depends on** | Working to a standard, every time, with no shortcuts that can't be defended |
| **Diagnosing faults on worn, real-world equipment** | Root-cause thinking on hardware that doesn't match the manual |
| **Hospital and long-term care sites** | Working safely and professionally in someone else's facility |
| **Quotes, invoices, service records** | Documentation, traceability and the cost side of engineering |
| **Clients and caregivers** | Turning a person's needs into a technical requirement |



FIG. 2.2 — Deck module ready to set beside an existing deck

> *The clients were the specification. A wheelchair that's fixed but not trusted hasn't been fixed.*

03 — FIELD PROJECT · 1 OF 2

# Accessibility Ramp Systems

Eight modular aluminum wheelchair ramps, configured to code and installed at clients' homes.



FIG. 3.1 — Finished straight run, Mar 2026

| CONTEXT | ROLE | PERIOD | METHODS |
|---|---|---|---|
| Action Medical Home Health | Junior Mobility Systems Technician | 2025 – 26 | Site survey · slope & run · modular layout · footings · install |

| 8 | 12–26 ft | 1 : 13 | ~1 day |
|---|---|---|---|
| RAMP SYSTEMS CONFIGURED | RANGE OF SPANS | ACHIEVED SLOPE, INSIDE THE 1 : 12 LIMIT | OF WORK PER SYSTEM, START TO FINISH |

## 01 BRIEF

Someone who uses a wheelchair or walker needs to get from the driveway to their front door without help. For many clients a ramp is the difference between leaving home on their own and not leaving at all.

The design space is narrow. Slope is capped at 1 : 12, the rise is fixed by the height of the threshold, and the footprint is whatever the property leaves once the driveway and existing steps are accounted for. The structure has to tie into what is already there (a deck, a set of steps, a porch), carry wheelchair, walker and caregiver traffic every day, and hold its geometry through Ontario freeze–thaw cycles. Every house resets the problem, and the person using the ramp can't wait for a second attempt.

## 02 CONSTRAINTS

| | |
|---|---|
| SLOPE | 1 : 12 maximum: 1 in of rise per 12 in of run |
| RISE | Measured grade to door threshold, every site |
| FOOTPRINT | Whatever the property allows, without blocking the driveway or access |
| INTERFACES | Existing decks and steps; gravel, soft ground and pavement |
| PARTS | Standard modules only: sections, turn platforms, landings, legs, handrails |
| USERS | Wheelchairs, walkers and caregivers, every day, in all weather |

## 03 THE DESIGN PROBLEM

Slope fixes length. Run is the rise multiplied by the slope ratio, so at the 1 : 12 limit a 36 in rise demands 432 in, or **36 ft**, of sloped deck before a single landing is added. The property has no say in that number; the only design freedom is in how the run is arranged in plan.

On the August 2025 switchback, the side yard couldn't hold anything near 36 ft in a straight line. Folding the run back on itself, two parallel legs joined by a full-width turn platform, converts a length problem into a width problem, and width was the one dimension the site could spare.

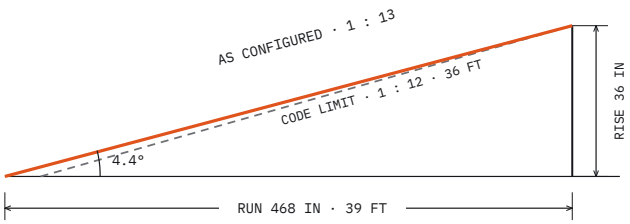



FIG. 3.2 — Same rise, two slopes. Going gentler than the limit costs 3 ft of run.

| | |
|---|---|
| Min. run at 1 : 12 | 36 in × 12 = 432 in = 36.0 ft |
| Run at 1 : 13 | 36 in × 13 = 468 in = 39.0 ft |
| Gradient | 7.7 % vs 8.3 % limit |
| Angle from horizontal | atan(1/13) = 4.40° |

Why go gentler than the limit? 1 : 12 is a maximum, not a target. Every bit of slope under it makes a ramp easier to climb under your own power and safer to come down, so when the property allows the extra run, the gentler layout wins.

## 04 PROCESS

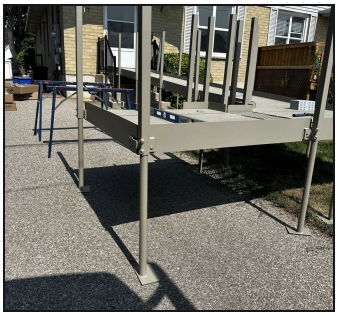
**01 Survey the site**
Measured the vertical rise from grade to the door threshold and walked the usable ground around the entrance: where the steps come down, how much driveway has to stay clear, what the ground is made of.

**02 Configure the layout**
Broke the required run into standard modules (straight sections, turn platforms and landings), then checked the slope the layout actually achieves against the slope selected on the layout sheet, and rearranged until it fit the property without crowding the driveway or blocking access.

**03 Plan the fabrication**
Turned each layout into a parts list and a staging plan, so modules, legs and handrails came off the truck in the order they went in. On the switchback: M1 × 9, M3 × 2, M4 × 1, M6 × 1, M7 × 2.

**04 Set footings and level**
Coordinated structural footing placement against what was already on the property: decks, steps, soft ground and gravel. Set the adjustable legs and levelled the frame before any decking went on.

**05 Install**
Installed the perforated deck sections, turn platforms and continuous handrails on both sides, so there is something to hold the full length of the ramp.



**3.4** Frame and legs, switchback



**3.5** Posts and landing frame, Mar 2026

| Not enough yard | FAILURE MODE 3.A |
|---|---|
| OBSERVED | A 36 in rise needs at least 36 ft of ramp at code slope. The side yard couldn't hold anything close to that in a straight line. |
| CONSTRAINT | Slope can't be steepened to save length. Run is fixed by the rise and the code. |
| RESOLVED | Fold it. Two parallel legs joined by turn platforms (module M3) pack about 39 ft of ramp into a 25.5 × 9.8 ft footprint. |
| VERIFIED | Layout sheet: actual slope 1 : 13 (≈ 4.4°), gentler than the 1 : 12 limit. |

| Ground that isn't level | FAILURE MODE 3.B |
|---|---|
| OBSERVED | Every footing sits at a different grade: gravel, soft ground, pavement, or the edge of an existing deck. |
| RISK | A ramp that follows the ground twists and changes slope along its length. |
| RESOLVED | Set each adjustable leg individually so the deck plane stays true from bottom to top, regardless of the ground under it. |

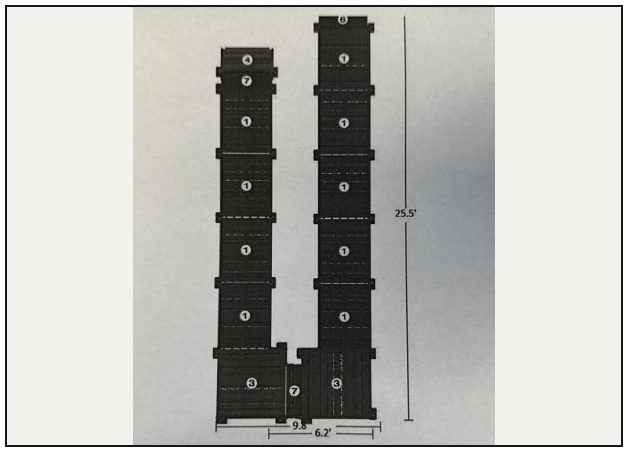



**FIG. 3.3** — Layout sheet, switchback install (Aug 2025)

## 05 OUTCOME

Clients got step-free access to their homes on ramps at or below the code slope limit. Eight systems spanning 12 to 26 ft, each close to a full day's work, from a tight switchback beside a house in August 2025 to a long straight run with a landing at a rural home in March 2026.

## 06 WHAT I TOOK AWAY

Engineering is judged by whether it works for the person using it, not by how it looks on the layout sheet. The numbers decide what's possible; the site and the client decide what's right.

## 07 REV. B

- Model the site and module layout in CAD before the first visit, so clearances and footing points are checked before anything is unloaded.
- Survey ground slope at every footing on the measuring visit, so leg heights can be preset.
- Turn the slope and run arithmetic into a calculator. A first version is on my portfolio site.

# Raspberry Pi Arcade Joystick

A working arcade controller built from a door stop, four nails and a block of wood, wired straight into a Raspberry Pi.

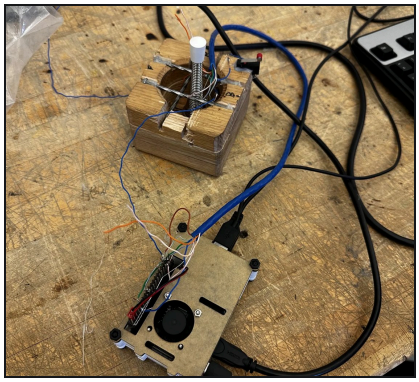


**FIG. 4.1** — Joystick and cased Pi, Jan 2026

| CONTEXT | ROLE | TIMELINE | METHODS |
|---|---|---|---|
| Independent project | Design, machining, wiring, configuration | 2 weeks set · < 1 used | Drill mill · GPIO · input mapping · debouncing |

| 4 + 1 | <1 wk | 0 | 2 |
|---|---|---|---|
| DIRECTIONS PLUS FIRE BUTTON | TO FINISH A TWO-WEEK BUILD | OFF-THE-SHELF JOYSTICK PARTS | GAMES USED TO TEST IT: TETRIS AND ALIENS |

## 01 BRIEF

Take an idea from concept to a working prototype using household parts instead of a kit. A store-bought joystick hides every interesting problem inside a plastic shell; building one from scratch meant solving the mechanical, electrical and software sides together, and seeing how a decision in one shows up in the others.

## 02 CONSTRAINTS

| | |
|---|---|
| PARTS | **What was on hand. No joystick kit, no microswitches** |
| TIME | **Two weeks allotted** |
| INTERFACE | **Must bind like a normal gamepad in an arcade front end** |
| FEEL | **Self-centring, equal throw in all four directions** |

## 03 HOW IT WORKS

The spring from a door stop is the stick: flexible enough to push, stiff enough to snap back to centre. Four nails stand around it, one per direction. Push the stick far enough and it touches a nail, which closes a circuit to one of the Pi's GPIO inputs. Electrically, the spring is the common contact and each nail is an independent input: touching a nail pulls that pin low, and releasing it lets the pull-up return it high. In effect, four normally-open switches made from hardware-store parts.

On the software side, an existing GPIO-to-gamepad mapping tool presents the pins to the arcade front end as a standard controller. My part was configuration: mapping each header pin to a direction (Fig. 4.4) and tuning the debounce timing until the stick felt right.

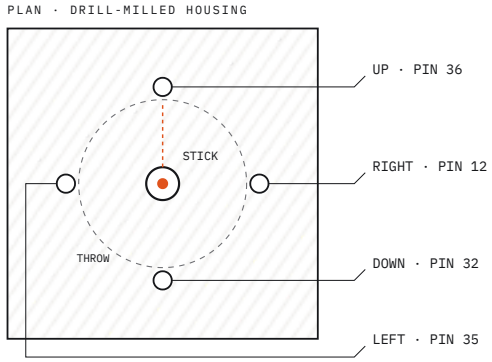



**FIG. 4.2** — Contact layout. Each nail sits where the stick meets it at the end of its throw, equally in every direction. Pin numbers are physical header pins, from the mapping in Fig. 4.4.

## 04 PROCESS

**01 Design around what was on hand**
Door-stop spring for a self-centring stick; nails as contacts. No kit, no microswitches.

**02 Lay out and machine the housing**
Measured, marked and clamped the wooden block, then drilled it on a drill mill so every nail landed at the same distance from the stick, following machine-shop safety practice.

**03 Wire it to the Pi**
Each direction and the fire button to its own GPIO input.

**04 Configure and tune the input layer**
Map pins to directions, bind the controls, tune debounce in real games.

| Contact bounce | FAILURE MODE 4.A |
|---|---|
| OBSERVED | One push of the stick registered as a burst of presses. In game, movement was jittery and laggy. |
| CAUSE | Metal-on-metal contacts don't close cleanly. The nail and spring chatter for a few milliseconds before settling, and the Pi reads fast enough to see every chatter as a separate press. |
| RESOLVED | Tuned the debounce timing in the mapping tool's configuration, so a state change only counts once the contact has held steady, and cleaned up the signal routing between the contacts and the header. |
| VERIFIED | Each push registered exactly once, with no visible lag, in play testing with Tetris and Aliens. |

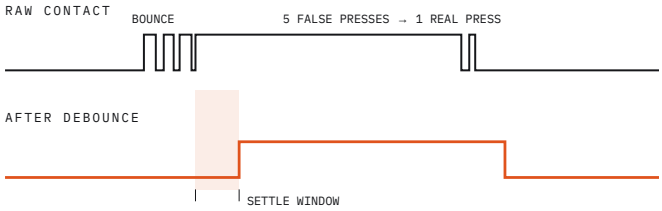



**FIG. 4.5** — Illustrative timing, not measured. A change is only accepted after the signal holds for the settle window.

## 05 DECISIONS AND TRADE-OFFS

| DECISION | INSTEAD OF | WHY |
|---|---|---|
| **Nail contacts** | Microswitches | On hand and free, as the brief required. The cost (bounce, wear) could be paid in software. |
| **Door-stop spring** | Custom spring and pivot | Self-centring for free, with a usable throw. |
| **Direct GPIO** | USB encoder board | No extra hardware, and it forces you to deal with the raw signal yourself. |
| **Drill-milled wood** | Hand drill or 3D print | Accurate, repeatable hole placement with the tools available. |

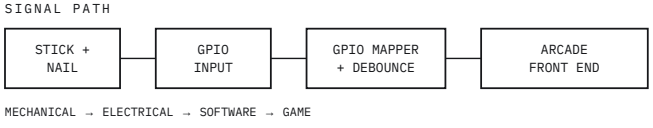
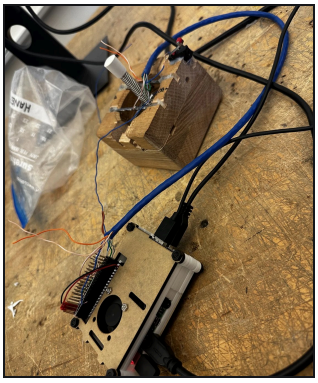
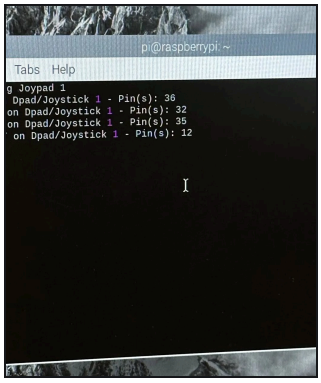



**FIG. 4.3** — One push crosses three domains before it moves a block.



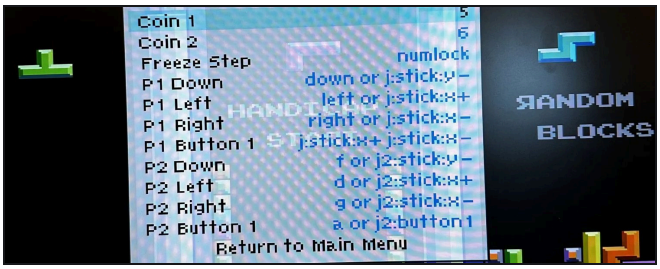
**4.6** Stick in the housing




**4.4** Direction → pin mapping

## 06 TWO WEEKS, DONE IN UNDER ONE

The project was scoped for two weeks and finished in under one. The time came from not working in series: the input mapping was tested on the Pi with the bare contacts while the housing was still being laid out, so by the time the stick was mounted, the software side was already known to work. The remaining time went into the part that only shows up once everything is assembled: how the controller actually feels to play.




**4.7** Controls bound in the arcade front end

## 07 OUTCOME

A fully working arcade controller, used to play Tetris and Aliens, finished in under a week on a two-week timeline. Every physical part, from the stick's spring to the contacts and housing, was designed and made by hand, and the software layer was configured and tuned around it.

> *Small changes in the software had a butterfly effect on how the whole system responded. Hardware and software can't be debugged in isolation.*

## 08 REV. B

- Swap the nails for microswitches, so the contacts stop wearing and bouncing.
- Model the housing in SolidWorks and cut it on a CNC or 3D printer for a repeatable fit.
- Add a second action button for two-button games.
- Replace the configured tool with my own Python input layer, for direct control over debouncing and timing.

05 — EMBEDDED + CONTROLS · 1 OF 2

# Arduino Rover

A two-motor rover in C++ that drives set shapes on its own and follows a line on the floor, rebuilt from scratch after its first controller was destroyed.

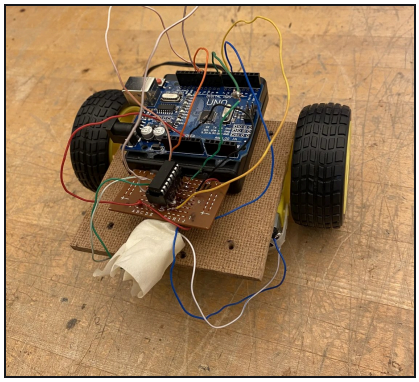


**FIG. 5.1** — The rebuilt rover, Jun 2024

| CONTEXT | ROLE | YEAR | METHODS |
|---|---|---|---|
| Partner build | Build, soldering, programming, testing | 2024 | C++ · PWM · IR sensing · soldering |

| 2 | <½ | 0 | 2 |
|---|---|---|---|
| DRIVING MODES: SHAPES AND LINE FOLLOWING | THE ORIGINAL BUILD TIME TO REBUILD FROM BARE PARTS | PARTS REPLACED TO FIX THE DRIFT | DOWNWARD IR SENSORS ON A HAND-SOLDERED BOARD |

## 01 BRIEF

Make a rover drive itself accurately on motors and wheels that were far from ideal. The two motors didn't run at the same speed and the wheels were substandard. Swapping parts wasn't an option, so every problem had to be solved in the build and in the code.

## 02 CONSTRAINTS

| | |
|---|---|
| DRIVE | **Two mismatched DC gearmotors, substandard wheels, front caster** |
| FEEDBACK | **None on the wheels: open loop. Position is only as good as the timing and calibration** |
| CONTROLLER | **Arduino Uno; pins can't supply motor current directly** |
| PARTS | **Fixed. Adapt the design, don't replace components** |

## 03 ARCHITECTURE

The Uno is the controller. Its outputs go to a motor driver on a hand-soldered perfboard: an L293D dual H-bridge, which lets the Uno's 5 V logic pins switch the current the motors actually need, in either direction, with PWM on the enable pins setting each motor's speed. Two reflective IR sensors on a second soldered board sit under the front of the chassis, a few millimetres off the floor, reading light versus dark.

Steering is differential: there's no steering mechanism, so the rover turns only by running one wheel faster than the other. That makes it simple to build and very sensitive to any mismatch between the two motors.

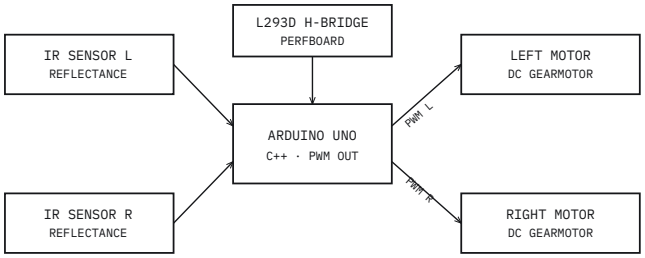



**FIG. 5.2** — System block diagram. Driver commands are drawn through the Uno for clarity.

## 04 TWO MODES, ONE CHASSIS

**Shape mode** runs timed sequences written in C++: drive straight for a set duration, turn for a set duration, repeat, so the rover traces squares and other geometric shapes on its own.

**Line-following mode** reads the two IR sensors. When one side sees the line, the code slows that side's motor and the rover steers back over it.

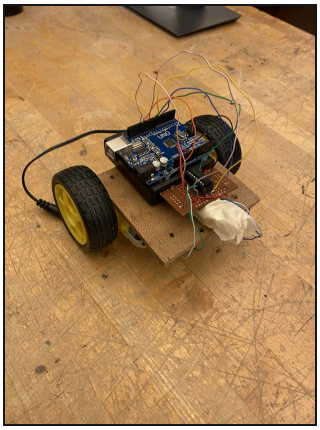
| LEFT SENSOR | RIGHT SENSOR | RESPONSE |
|---|---|---|
| **Floor** | Floor | Both motors on: drive straight |
| **Line** | Floor | Slow left motor: steer left, back onto the line |
| **Floor** | Line | Slow right motor: steer right, back onto the line |

| Dead board, full restart | FAILURE MODE 5.A |
|---|---|
| OBSERVED | Partway through the build, the first Arduino was fried. The board was dead, and with it the build: we had to start over. |
| CAUSE | During testing the rover was hooked up to a battery tester instead of its normal supply. The tester delivered far more power than the circuit was built to take: the motors ran at runaway speed, and the board burned out under the over-voltage. |
| RESOLVED | Started again from bare parts on a new board. Knowing the build the second time, we finished the full rebuild in **less than half the time** the first attempt took. |
| PROTECTED | On the rebuild, every connection was checked against the plan and the supply polarity verified before each power-up, and motor power was kept off the logic side. |

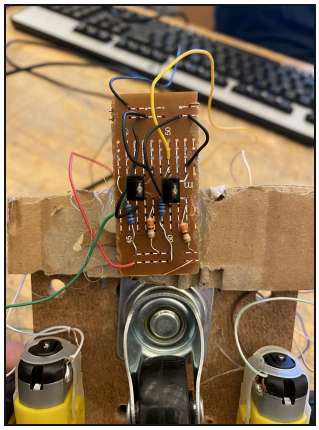
## 05 WHY THE REBUILD WAS FASTER

The first build was also the learning: working out where each wire went, which order to assemble in, and how to lay out the perfboard. None of that had to be rediscovered. The second build was execution against a design we already understood, which is why it took less than half the time.

It changed how I think about a setback. Losing the hardware didn't lose the work, because the work was the understanding. Being able to rebuild a system quickly matters as much as building it right the first time.



**5.3** Uno on the hardboard chassis



**5.4** Hand-soldered IR sensor board, underside

| Drift | FAILURE MODE 5.B |
|---|---|
| OBSERVED | Over repeated test runs the rover wandered off its path, and the shapes didn't close. |
| CAUSE | Equal PWM never gave equal speed. The two motors had different outputs, and gearbox slop and wheel friction added their own bias. |
| RESOLVED | Calibrated each motor's PWM duty cycle separately, then tuned turn timing, acceleration delays and the movement sequences run by run until both sides matched. |
| VERIFIED | Shapes came out consistent across repeated runs, on the same motors and wheels it started with. |

## 06 WHY SMALL MISMATCHES MATTER

With differential steering, the rover's turning rate is set by the difference between its wheel speeds divided by the distance between the wheels. Any difference at all becomes a steady turn, and with no encoders to notice it, the error builds up for as long as the rover drives. A mismatch too small to see on the bench becomes a square that doesn't close.

| | |
|---|---|
| Turn rate | $\omega = (v_R - v_L) / \text{track width}$ |
| Heading error | $\Delta\theta = \omega \cdot t$, growing every second |
| Correction applied | separate PWM duty per motor |

Calibration doesn't remove the cause, it cancels it: each motor gets the duty cycle that makes its real speed match the other's. That's the limit of an open-loop system, and the reason encoders are first on the Rev. B list.

## 07 OUTCOME

A rover that draws its shapes consistently and follows a line, on the same imperfect motors and wheels it started with, delivered despite losing the first build entirely.

> *When the parts aren't ideal, fix it in the design and the code. When the hardware is lost, the understanding isn't.*

## 08 REV. B

- Wheel encoders for closed-loop speed control instead of fixed PWM offsets.
- PID steering in line-following mode for smoother, faster tracking.
- A fused, separate motor supply to protect the logic board.

06 — HARDWARE · TEAM BUILD

# ESP32 Walkie-Talkie

A working push-to-talk walkie-talkie, built from a bag of parts against the clock. Our team finished first.

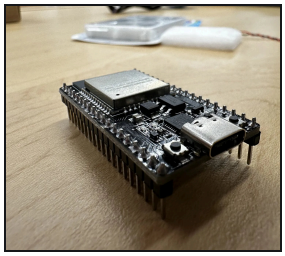


**FIG. 6.1** — ESP32 development board

| CONTEXT | ROLE | YEAR | METHODS |
|---|---|---|---|
| Workshop competition | Team member: circuit build, testing | 2025 | ESP32 · ESP-NOW · breadboarding · audio amp |

| 1st | 1 | 1 |
|---|---|---|
| TEAM TO A WORKING MODEL | BREADBOARD, NO SOLDERING | BUTTON: HOLD TO TALK, RELEASE TO LISTEN |

## 01 BRIEF

A timed workshop competition: turn a bag of parts into a walkie-talkie that actually works, before anyone else. Every team started with the same parts and the same clock. Nothing counted except a working model.

## 02 APPROACH

**01 Build around the ESP32**
The ESP32 pairs a fast processor with a 2.4 GHz radio, so one chip samples the audio, packs it into packets and transmits it.

**02 Drive the speaker**
A GPIO pin can't source the current a speaker needs, so received audio runs through a small amplifier board to a compact speaker.

**03 Add push-to-talk**
One button switches the unit between talking and listening: hold to talk, release to hear the other side.

| The clock | FAILURE MODE 6.A |
|---|---|
| RISK | A hard time limit, with every team racing for the same finish line. A fix that took minutes could cost the win. |
| DECISION | No soldering. With the whole circuit on one breadboard, every correction was a wire moved and a re-test in seconds. |
| RESULT | First team to a working model. |

## 04 REV. B

- Move to a soldered perfboard with a battery, so it works away from a laptop.
- Design a handheld enclosure in Fusion 360 and 3D-print it.

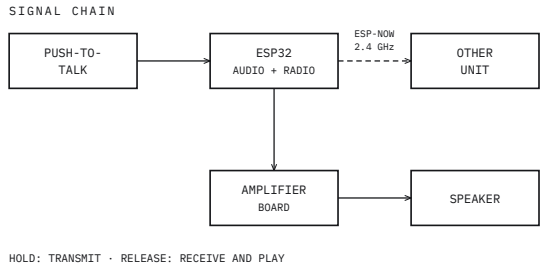



**FIG. 6.2** — Signal chain of one unit.

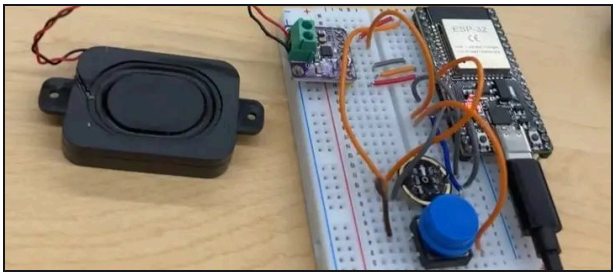


**FIG. 6.3** — The finished prototype: ESP32, amplifier, speaker and push-to-talk button on one breadboard. A video of it working is on the portfolio site.

## 03 HOW THE LINK WORKS

Builds like ours talk over **ESP-NOW**, Espressif's connectionless peer-to-peer protocol. It uses the ESP32's Wi-Fi radio in the 2.4 GHz band, the same band as Bluetooth, but needs no router, pairing or network: the talking unit broadcasts short packets of audio samples (up to 250 bytes each) and every unit on the same channel receives them, much like a handheld radio shares a frequency.

Near-zero setup and low latency suit a race against the clock. The trade-off is unacknowledged delivery: a dropped packet is a brief click, not a resend.

> *Under a deadline, the design that's fastest to change beats the one that's neatest on paper.*

# Personal Finance & Portfolio Model

A spreadsheet model for real decisions about Canadian registered accounts, an ETF, and loan repayment.

| CONTEXT | ROLE | YEAR | TOOLS |
|---|---|---|---|
| Personal project | Modelling and analysis | 2025 | Microsoft Excel · Monarch Money |

| 3 | 3 | 3 |
|---|---|---|
| REGISTERED ACCOUNT TYPES COMPARED: TFSA, RRSP, FHSA | RETURN HORIZONS PER FUND: 1, 3 AND 5 YEARS | DECISIONS MODELLED: ACCOUNTS, FUND, REPAYMENT |

## 01 BRIEF

Answer practical money questions with a model instead of a guess. Which mix of TFSA, RRSP and FHSA contributions comes out ahead over time? Is a Canadian dividend ETF worth its fees? How much does a different repayment schedule really change the interest paid?

It's on an engineering portfolio because it's the same discipline: define the inputs, model the system honestly, then compare scenarios on equal terms.

## 02 MODEL STRUCTURE

**01 Registered accounts**
Projected long-term growth across TFSA, RRSP and FHSA with annual contributions, compound growth, asset allocation and shifts in marginal tax bracket.

**02 Fund suitability**
Compared XEI, a Canadian equity dividend ETF, on management expense ratio, dividend yield and trailing 1-, 3- and 5-year total returns.

**03 Repayment schedules**
Built amortization models comparing alternative loan repayment schedules: principal reduction, timeline and cumulative interest under different rates and cash flows.

### Unequal tax treatment — FAILURE MODE 7.A

| | |
|---|---|
| PROBLEM | TFSA, RRSP and FHSA are taxed differently going in and coming out, so their balances can't be compared at face value. A larger RRSP balance can be worth less after tax than a smaller TFSA. |
| RESOLVED | Modelled each account with its own tax treatment and the shift in marginal bracket over time, so all three are compared on the same after-tax footing. |

## 03 THE CORE ARITHMETIC

| | |
|---|---|
| Future value, yearly C | $FV = C \cdot ((1 + r)^n - 1) / r$ |
| Level payment on P | $M = P \cdot i / (1 - (1 + i)^{-N})$ |
| Interest, period k | $I_k = i \cdot balance_{k-1}$ |
| ETF net return | total return − MER |

r: annual growth rate · n: years · i: periodic interest rate · N: number of payments.

## 04 OUTCOME

A reusable model that turns each of these questions into numbers that can be compared side by side.

## 05 REV. B

- Replace the single growth rate with a range of return scenarios.
- Rebuild the model in Python so scenarios can be swept automatically.